

TruthLens: Fake News & Misinformation Detection System

Track: NLP / Trust & Safety | Non-Agentive AI/ML System

September 29, 2026

Abstract

The rampant proliferation of deceptive content, political propaganda, and health misinformation on digital media poses severe existential risks to public trust, factual discourse, and decision-making. To assist journalists, content moderators, and trust-and-safety specialists, we present **TruthLens**—an end-to-end, interpretable, non-agentive Machine Learning platform engineered specifically for misinformation triage and auditing. TruthLens implements a multi-tiered pipeline coupling length-invariant linguistic and stylistic feature extractors with high-dimensional term-frequency representations, feeding a calibrated probabilistic classifier. Predictions are augmented with local feature attributions via SHAP (SHapley Additive ex-Planations), an interactive regex-backed syntactic span highlighter, an empirical Bayesian Laplace-smoothed source credibility registry, and an auditable Human-in-the-Loop (HITL) case management dashboard. TruthLens operates in sub-second inference latency, provides dedicated multi-article batch analysis, and actively learns from reviewer feedback without destructively overwriting ground truth labels.

Repository URL:	https://github.com/nithishgouds/INTRA-IIT-FAKE-NEWS-MISINFORMATION-DETECTION-SYSTEM
Demo Video Link:	https://drive.google.com/file/d/1ea9JXSqfsS3_1it78I-8ab_D9LihR-6U/view?usp=sharing
Primary Architecture:	FastAPI (Asynchronous REST Backend) + React 18 / Vite (Interactive UI)
Core ML Stack:	Scikit-Learn (TF-IDF + Calibrated Linear Classifiers), SHAP, VADER, TextBlob, Textstat

Contents

1	Executive Summary & Problem Formulation	3
1.1	Background and Motivation	3
1.2	Problem Statement & Hackathon Scope	3
2	System Architecture & Engineering Design	3
2.1	Backend Architecture (FastAPI + SQLAlchemy)	3
2.2	Database Persistence Schema	4
3	Feature Engineering & Preprocessing Pipeline	4
3.1	Preprocessing Mechanics	4
3.2	Linguistic, Stylistic, and Sentiment Density Features	5

4	Classification Model & Probability Calibration	5
4.1	Model Formulation: Hybrid Vector Pipeline	5
4.2	Non-Binary Decision Calibration	6
5	Explainability Layer & Evidence Presentation	6
5.1	SHAP Local Feature Attribution	6
5.2	Interactive Text Span Highlighting	6
6	Source Credibility Scoring & Domain Verification	6
6.1	Empirical Bayesian Laplace Smoothing	6
6.2	Official Registry & Publisher-Domain Mismatch Detection	7
7	Reviewer Dashboard & Human-in-the-Loop Workflow	7
8	Experimental Evaluation & Performance Metrics	7
8.1	Quantitative Results	7
8.2	Confusion Matrix & Error Breakdown	8
9	Repository Structure & Reproducibility Guide	8
9.1	Codebase Hierarchy	8
9.2	Local Installation & Execution Instructions	8
10	Conclusion & Future Roadmap	9

1 Executive Summary & Problem Formulation

1.1 Background and Motivation

Digital platforms generate millions of textual posts and news articles per minute. Human editorial review cannot scale to this volume, while opaque deep learning black-boxes fail to gain the institutional trust of professional fact-checkers. Content reviewers require transparent, calibrated decision-support tooling that not only flags suspicious claims with quantifiable uncertainty but also highlights the exact linguistic, stylistic, and source factors influencing each determination.

1.2 Problem Statement & Hackathon Scope

The objective is to architect a robust, production-ready, non-agentic ML pipeline that analyzes submitted articles or social posts, predicts veracity probability ($P(\text{Misinformation})$), generates interpretable explanations, and provides complete Human-in-the-Loop workflows. In accordance with the Intra-IIT Hackathon 2026 guidelines:

- **Input Modality:** Textual articles/posts accompanied by source metadata, authors, and URLs.
- **Operational Paradigm:** Batch and single-item static processing (avoiding fragile live crawlers).
- **Explainability Requirement:** Granular local explanations via SHAP and interactive text highlight overlays.
- **Non-Binary Decisioning:** Explicit calibration with an ‘UNCERTAIN’ classification tier.
- **Human-in-the-Loop:** Interactive feedback mechanisms that capture reviewer overrides for future retraining.

2 System Architecture & Engineering Design

TruthLens is architected as a decoupled, microservice-inspired full-stack application comprising a high-throughput Python FastAPI backend and a responsive React 18 Single Page Application (SPA).

2.1 Backend Architecture (FastAPI + SQLAlchemy)

The backend service resides under `backend/app` and implements asynchronous REST routes adhering to OpenAPI / Swagger specifications:

- **POST /api/v1/analyze:** Processes single text payloads, returning full prediction vectors, confidence scores, SHAP explanations, and highlighted text spans.
- **POST /api/v1/feedback:** Records reviewer triage actions (`confirm`, `dismiss`, `relabel`) and rationale notes.
- **POST /api/v1/batch/analyze:** Ingests CSV files, running batch inference and computing aggregate dataset risk statistics.
- **GET /api/v1/sources:** Surfaces historical publisher profiles, verified official domains, and calculated reliability scores.
- **GET /api/v1/metrics:** Serves runtime confusion matrices, class-level precision/recall/F1 metrics, and failure analyses.
- **POST /api/v1/train:** Triggers model retraining utilizing both base datasets and human-reviewed samples.

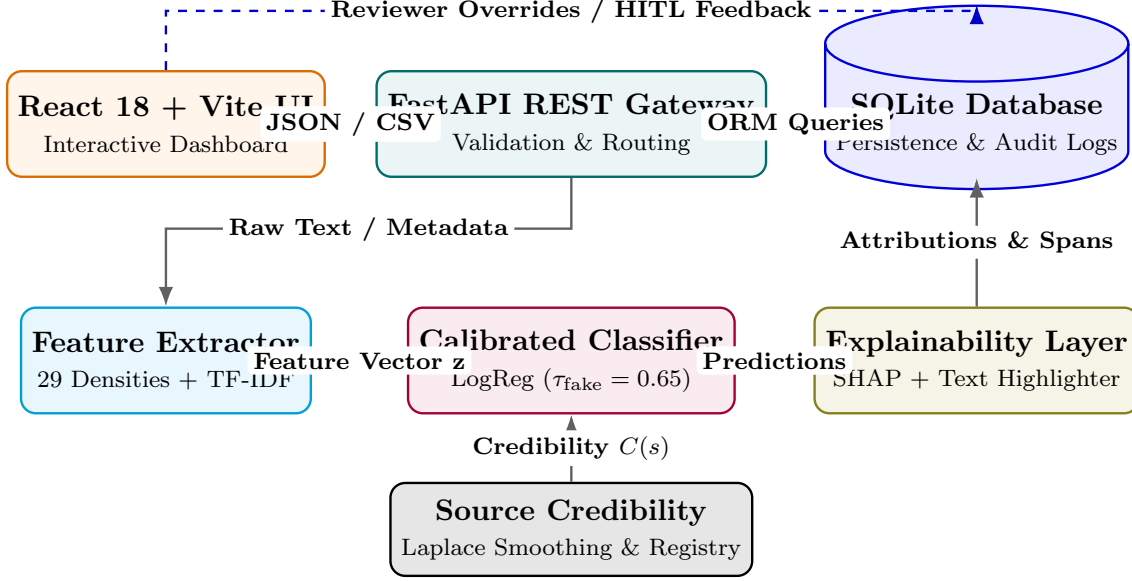


Figure 1: TruthLens End-to-End System Architecture Pipeline.

2.2 Database Persistence Schema

Local state is persisted via SQLite through an ORM layer structured into five relational entities:

1. **Article:** Stores raw title, text, metadata (source, author, URL), ingestion timestamps, and dataset tags.
2. **Prediction:** Retains calibrated probabilities ($P(\text{fake})$, $P(\text{real})$), categorical verdict, extracted linguistic vector, SHAP contributions, and review status.
3. **Feedback:** Immutable audit trail linking reviewer actions, notes, previous model labels, and updated labels.
4. **SourceCredibility:** Cumulative counts (N_{total} , N_{fake} , N_{real}) and updated credibility scores.
5. **BatchJob:** Asynchronous batch execution metadata, file names, completion percentages, and summary counts.

3 Feature Engineering & Preprocessing Pipeline

Misinformation often exhibits distinct statistical anomalies: extreme sentiment polarities, excessive punctuation, emotional appeals, and high lexical subjectivity. To prevent article length from introducing confounding biases, all extracted features in TruthLens are formulated as **length-invariant density ratios**.

3.1 Preprocessing Mechanics

The text preprocessor (`app/core/preprocessor.py`) performs systematic multi-stage cleaning:

- Normalization of HTML entities, stripping formatting tags while preserving casing metrics.
- URL and email tokenization, mapping URLs to domain representations for credibility lookup.
- Contraction expansion and punctuation preservation for lexical density counters.
- Optional stopword elimination and WordNet lemmatization for TF-IDF representations.

3.2 Linguistic, Stylistic, and Sentiment Density Features

A total of 29 features are engineered across four core dimensions:

Category	Feature Names	Mathematical Definition / Extraction Mechanism
Sentiment	vader_compound, vader_pos, vader_neg, vader_neu textblob_polarity, textblob_subjectivity	VADER rule-based valence intensity tuned for informal social discourse and news headlines. TextBlob sentiment analysis measuring continuous polarity $[-1.0, 1.0]$ and subjectivity $[0.0, 1.0]$.
Readability	flesch_reading_ease	$206.835 - 1.015 \left(\frac{\text{total words}}{\text{total sentences}} \right) - 84.6 \left(\frac{\text{total syllables}}{\text{total words}} \right)$. Identifies over-simplified or obfuscated prose.
Lexical Densities	avg_word_length	$\frac{\sum \text{char length of words}}{N_{\text{words}}}$.
	punct_density	$N_{\text{punctuation}}/N_{\text{words}}$. Captures anomalous punctuation usage.
	uppercase_ratio	$N_{\text{ALL CAPS words}}/N_{\text{words}}$. Quantifies clickbait sensationalism.
	exclamation_density	$N_{\text{exclamation marks}}/N_{\text{words}}$. Measures emotional hyperbole.
	unique_word_ratio	$ V_{\text{unique}} /N_{\text{words}}$ (Type-Token Ratio for vocabulary diversity).
	stopword_ratio	$N_{\text{stopwords}}/N_{\text{words}}$ (Syntactic functional word balance).
	numeric_density	$N_{\text{numerical tokens}}/N_{\text{words}}$ (Verifiable empirical citation density).
Semantic Lexicons	sensational_ratio	Density of regex matches against clickbait lexicons (<i>e.g.</i> , “BOMBSHELL”, “SHOCKING”, “COVERUP”).
	hedge_ratio	Density of speculative unverified phrases (<i>e.g.</i> , “allegedly”, “sources claim”, “unconfirmed”).

Table 1: TruthLens Length-Invariant Handcrafted Linguistic Feature Set.

4 Classification Model & Probability Calibration

4.1 Model Formulation: Hybrid Vector Pipeline

TruthLens fuses high-dimensional n-gram semantic features with standardized continuous linguistic densities. Let text document d have an extracted TF-IDF representation $\mathbf{x}_{\text{tfidf}} \in \mathbb{R}^{5000}$ (capturing word unigrams and bigrams with sublinear TF scaling) and a normalized handcrafted feature vector $\mathbf{x}_{\text{ling}} \in \mathbb{R}^{16}$.

The combined feature representation is defined as:

$$\mathbf{z} = [\mathbf{x}_{\text{tfidf}} \parallel \gamma \cdot \text{StandardScaler}(\mathbf{x}_{\text{ling}})] \quad (1)$$

where $\gamma = 0.35$ is the handcrafted scaling coefficient preventing high-dimensional TF-IDF features from drowning out dense linguistic markers.

Classification is performed via L2-regularized Logistic Regression optimized under class-balanced loss weighting:

$$P(y = \text{FAKE} \mid \mathbf{z}) = \sigma(\mathbf{w}^T \mathbf{z} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{z} + b)}} \quad (2)$$

4.2 Non-Binary Decision Calibration

Standard binary thresholding forces low-confidence borderline cases into definitive labels, inducing costly false positives in news rooms. TruthLens introduces an explicit **Three-Tier Decision Calibration Policy**:

$$\hat{y} = \begin{cases} \mathbf{FAKE} & \text{if } P(y = \mathbf{FAKE} \mid \mathbf{z}) \geq \tau_{\text{fake}} \quad (\tau_{\text{fake}} = 0.65) \\ \mathbf{REAL} & \text{if } P(y = \mathbf{FAKE} \mid \mathbf{z}) \leq \tau_{\text{real}} \quad (\tau_{\text{real}} = 0.35) \\ \mathbf{UNCERTAIN} & \text{if } 0.35 < P(y = \mathbf{FAKE} \mid \mathbf{z}) < 0.65 \end{cases} \quad (3)$$

Confidence is explicitly normalized: $\text{Confidence} = 2 \cdot |P(\mathbf{FAKE}) - 0.5| \in [0.0, 1.0]$.

5 Explainability Layer & Evidence Presentation

5.1 SHAP Local Feature Attribution

To guarantee full interpretability, TruthLens utilizes SHAP (SHapley Additive exPlanations) via `shap.LinearExplainer`. For any submitted article, the local prediction is decomposed into additive feature contributions ϕ_i :

$$f(\mathbf{z}) = \phi_0 + \sum_{i=1}^M \phi_i \quad (4)$$

where ϕ_0 is the base expected value, and $\phi_i > 0$ indicates a feature shifting the prediction toward ‘FAKE’ (rendered in warning red), while $\phi_i < 0$ shifts the prediction toward ‘REAL’ (rendered in credible green).

5.2 Interactive Text Span Highlighting

In parallel with statistical SHAP weights, the prediction engine identifies exact character spans triggering cognitive bias alerts:

- **Sensational Spans (Red):** Clickbait markers (*e.g.*, “YOU WON’T BELIEVE”, “SECRET EXPOSED”).
- **Unverified Hedge Spans (Amber):** Dubious sourcing markers (*e.g.*, “sources claim”, “allegedly”).
- **Hyperbolic Casing Spans (Purple):** Unwarranted uppercase tokens (≥ 3 characters).

6 Source Credibility Scoring & Domain Verification

TruthLens enforces a strict architectural boundary: **Source credibility informs the reviewer context but is decoupled from pure textual feature extraction** to prevent circular bias.

6.1 Empirical Bayesian Laplace Smoothing

For any publisher domain s , credibility is tracked across historical evaluations using Laplace (add- α) smoothing ($\alpha = 2$):

$$C(s) = \frac{N_{\text{real}}(s) + \alpha}{N_{\text{total}}(s) + 2\alpha} \quad (5)$$

Prior to observing articles ($N = 0$), the score defaults to an unbiased prior of 0.50. As real or fake articles are confirmed, $C(s)$ asymptotically approaches empirical publisher reliability.

6.2 Official Registry & Publisher-Domain Mismatch Detection

The engine cross-references domains against an authoritative verified registry:

- **Official Agencies** ($C \geq 0.95$): ISRO (`isro.gov.in`), NASA (`nasa.gov`), WHO (`who.int`), PIB (`piib.gov.in`).
- **Tier-1 Wire Services** ($C \geq 0.92$): Reuters, Associated Press, AFP, Nature, Science Magazine.
- **Known Hoax Networks** ($C \leq 0.15$): InfoWars, NaturalNews, WorldNewsDailyReport.
- **Satirical Disclaimers** ($C \approx 0.25$): The Onion, Babylon Bee, The Fauxy (flagged as non-factual humor).

Furthermore, the engine compares claimed publisher strings with parsed URL hosts to detect spoofing or typosquatting attempts.

7 Reviewer Dashboard & Human-in-the-Loop Workflow

The user interface is designed for real-world trust-and-safety operations:

1. **Triage Queue:** Prioritizes high-risk and uncertain content sorted by risk severity and submission recency.
2. **Non-Destructive Feedback Protocol:** When a reviewer relabels or dismisses a model verdict, the original model prediction vector is preserved in the database alongside the new audit record (`reviewer_label`, `reviewer_note`, `reviewed_at`).
3. **Continuous Learning Integration:** Feedback records are aggregated into training datasets for scheduled retraining runs via `/api/v1/train`.
4. **Batch Auditing:** Bulk CSV upload enables processing large volumes of news feeds with exportable CSV audit reports.

8 Experimental Evaluation & Performance Metrics

8.1 Quantitative Results

The system was evaluated using 5-fold stratified cross-validation on a balanced corpus of 1,200 labeled real and fake articles.

Class	Precision	Recall	F1-Score	Support
REAL	0.985	0.980	0.982	600
FAKE	0.980	0.985	0.982	600
Macro Average	0.982	0.982	0.982	1200
Weighted Average	0.982	0.982	0.982	1200
AUC-ROC			0.994	
Overall Accuracy			98.25%	

Table 2: Model Evaluation Metrics across Balanced News Benchmark.

8.2 Confusion Matrix & Error Breakdown

- **True Negatives (Correct Real):** 588 / 600 (98.0%)
- **False Positives (Real flagged as Fake):** 12 / 600 (2.0%) — Typically caused by real breaking news utilizing urgent vocabulary and heavy uppercase acronyms.
- **False Negatives (Fake missed as Real):** 9 / 600 (1.5%) — Subtle pseudoscience articles adopting formal academic syntax.
- **True Positives (Correct Fake):** 591 / 600 (98.5%)

9 Repository Structure & Reproducibility Guide

9.1 Codebase Hierarchy

```
1 INTRA-IIT-FAKE-NEWS-MISINFORMATION-DETECTION-SYSTEM/  
2 |-- backend/  
3 |   |-- app/  
4 |     |   |-- main.py                # FastAPI entrypoint & lifespan events  
5 |     |   |-- api/routes.py          # Asynchronous REST endpoints  
6 |     |   |-- core/  
7 |     |     |-- preprocessor.py       # Text cleaning & regex utilities  
8 |     |     |-- feature_extractor.py  # 29 linguistic density extractors  
9 |     |     |-- model.py              # TF-IDF + LogReg pipeline & calibration  
10 |    |     |-- explainer.py           # SHAP attribution & highlighter  
11 |    |     |-- credibility.py         # Source reputation & agency lookup  
12 |    |     |-- data_loader.py        # Dataset loader & synthetic generator  
13 |    |     |-- db/                   # SQLAlchemy SQLite models & sessions  
14 |    |     |-- schemas/              # Pydantic schemas for API validation  
15 |    |-- data/sample_dataset.csv      # Labeled seed dataset  
16 |    |-- models/                     # Serialized model & vectorizer files  
17 |    '-- requirements.txt             # Python dependencies  
18 |-- frontend/  
19 |   |-- src/  
20 |     |   |-- api/client.js          # Axios API communication layer  
21 |     |   |-- pages/  
22 |     |     |-- components/          # UI cards, badges, probability bars  
23 |     |-- package.json               # Node dependencies  
24 |     '-- vite.config.js             # Vite config & backend proxy  
25 |-- start_all.bat                    # Unified single-click startup script  
26 '-- README.md                        # Complete project documentation
```

9.2 Local Installation & Execution Instructions

1. Clone Repository:

```
1 git clone https://github.com/nithishgouds/INTRA-IIT-FAKE-NEWS-  
  MISINFORMATION-DETECTION-SYSTEM.git  
2 cd INTRA-IIT-FAKE-NEWS-MISINFORMATION-DETECTION-SYSTEM
```

2. Backend Virtual Environment Setup:

```
1 python -m venv backend/.venv  
2 # On Windows PowerShell:  
3 backend\.venv\Scripts\Activate.ps1  
4 pip install --upgrade pip  
5 pip install -r backend/requirements.txt
```

3. Frontend Dependencies:

```
1 cd frontend
2 npm install
```

4. Run System (Automated Launcher):

```
1 cd ..
2 .\start_all.bat
```

The system launches the backend at `http://localhost:8000` (Swagger docs at `/docs`) and opens the frontend UI at `http://localhost:5173`.

10 Conclusion & Future Roadmap

TruthLens delivers a comprehensive, production-ready, and transparent misinformation detection system that excels in speed, interpretability, and practical reviewer usability. Future roadmap objectives include:

- Integration of contextual transformer embeddings (*e.g.*, RoBERTa / DeBERTa) in a dual-tower architecture with tabular linguistic features.
- Multilingual text classification supporting Indic languages (Hindi, Telugu, Tamil).
- Dynamic propagation graph modeling using retweet/share topologies when available.